A

LABORATORY FILE

On

OOP EXPERIMENT_6

School of Computer Science, UPES, Dehradun.

B.TECH. - IV Semester

Aug. – Dec. (2025)

Submitted to:

Prof. Kalluri Shareef Babu

SoCS, UPES

Submitted by:

Name: Anmol

Batch: 20

SAP ID: 5900011794

Theory

Inheritance

- Inheritance allows one class to acquire properties and methods of another class.
- Implemented using extends keyword.
- Promotes code reusability and hierarchical classification.
- Demonstrated in programs like Person Student, Vehicle to Car/Bike, Employee to Manager to Director.

Types of Inheritance

- Single inheritance: one subclass inherits from one superclass.
- Hierarchical inheritance: multiple subclasses inherit from same superclass.
- Multilevel inheritance: inheritance chain across multiple levels.
- Examples are implemented using Person, Vehicle, and Employee class hierarchies.

Method Overriding

- Subclass provides its own implementation of a method defined in superclass.
- Method signature (name + parameters) must be same.
- Enables runtime polymorphism.
- Used in display(), area(), interest() methods.

Runtime Polymorphism

- Method call is resolved at runtime based on object type.
- Achieved using method overriding and superclass reference.
- JVM decides which method to execute.
- Demonstrated using Shape, Animal, and Bank classes.

Dynamic Method Dispatch

- Superclass reference variable refers to subclass object.
- Method execution depends on actual object assigned.
- Often used with conditional logic or user input.

- Demonstrated using Shape selection programs.

super Keyword

- Used to call superclass method or constructor.
- super.method() calls parent class version of method.
- Helps extend functionality instead of replacing it.
- Must be explicitly written.

Abstract Classes

- Declared using abstract keyword.
- Cannot be instantiated directly.
- May contain abstract and non-abstract methods.
- Subclasses must override abstract methods.
- Used in Employee, Vehicle, Bank, Appliance classes.

Interfaces

- Used to define completely abstract behavior.
- Methods are implicitly public and abstract.
- Implemented using implements keyword.
- Supports multiple inheritance.
- Used in Shape, Payment, Drawable interfaces.

Multiple Inheritance using Interfaces

- Java does not support multiple inheritance with classes.
- Achieved using multiple interfaces.
- A class can implement more than one interface.
- Example: AllInOnePrinter implements Printer and Scanner.

Interface-based Polymorphism

- Interface reference can refer to objects of implementing classes.
- Enables flexible and dynamic method execution.
- Demonstrated using Shape, Payment, Drawable implementations.

Array of Objects with Polymorphism

- Array of superclass type can store different subclass objects.
- Looping over array invokes correct overridden methods.
- Demonstrates runtime polymorphism.
- Used in Animal, Bank, and Appliance programs.

Dynamic Binding

- Method call is resolved during runtime, not compile time.
- Based on actual object type.
- Core concept behind runtime polymorphism.

Type Checking and Casting

- instanceof operator is used to check object type.
- Required when calling subclass-specific methods.
- Typecasting is used to access subclass methods.
- Demonstrated in Account-based programs.

Constructor Usage in Inheritance

- super() is used to call superclass constructor.
- Helps initialize inherited data members.
- Avoids duplication of initialization logic.

Input Handling Used

- Scanner is used for user input.
- nextLine(), nextInt(), nextDouble() methods are used.
- Input controls object creation and method execution.

Output Functions

- System.out.print() and System.out.println() are used.
- Used to display results and behavior of overridden methods.

Object Creation and Method Invocation

- Objects are created using new keyword.
- Superclass references can hold subclass objects.
- Method calls are resolved dynamically at runtime.

INDEX

Number	Problem	Page No.
Th1	Why is Runtime Polymorphism also called "Late Binding"?	7
Th2	Can you override a static method in Java? Why or why not?	7
Th3	What is the difference between this() and super() calls?	7
Th4	What happens if you try to override a final method?	8
Th5	Explain Covariant Return Types in the context of method overriding.	8
Th6	Is it possible to perform multiple inheritance using classes in Java?	8
Th7	Can abstract classes have constructors?	9
Th8	What are Default methods?	9
Th9	Is 'final' allowed for abstract classes?	10
Th10	Difference between extends & implements.	10
Th11	What are Marker Interfaces?	10
Th12	Can an interface extend another interface?	11
Th13	Can we define static methods in interfaces?	11
Th14	Define 'Loose Coupling'.	12
Exp6.1	Write a Java program to demonstrate single inheritance using class Person and subclass Student Override a method display() in the subclass	12
Exp6.2	Develop a Java program to implement runtime polymorphism using class Shape and subclass Circle. Override the method area().	13
Exp6.3	Write a Java program to demonstrate hierarchical inheritance using class Vehicle and subclasses Car and Bike.	14
Exp6.4	Implement a Java program to demonstrate method overriding using class Bank and subclasses SBI and HDFC to compute interest rates	15
Exp6.5	Write a Java program to demonstrate multilevel inheritance using classes Employee → Manager → Director.	17
Exp6.6	Develop a Java program to demonstrate runtime polymorphism using an array of objects. Use class Animal and subclasses Dog and Cat.	19

Exp6.7	**Write a Java program to demonstrate dynamic method dispatch using user input. Create base class Shape and subclasses Circle and Square.**	**20**
Exp6.8	**Design a Java program to demonstrate method overriding using the super keyword. Create base class Vehicle and subclass Car where overridden method calls parent method.**	**22**
Exp6.9	**Create an abstract class Employee with abstract method calculateSalary(). Implement subclasses Manager and Developer and override the method.**	**23**
Exp6.10	**Design an interface Shape with method area(). Implement classes Circle, Rectangle, and Triangle.**	**24**
Exp6.11	**Create an abstract class Vehicle with abstract method start() and Implement subclasses Car and Bike demonstrating runtime polymorphism**	**26**
Exp6.12	**Write a Java program to demonstrate interface-based runtime polymorphism using Payment interface and classes CreditCard and UPI.**	**28**
Exp6.13	**Create an interface Drawable and demonstrate hierarchical implementation using classes Circle and Square.**	**30**
Exp6.14	**Design an abstract class Bank with method getInterestRate(). Implement subclasses SBI, HDFC, and ICICI. Demonstrate runtime polymorphism using array of objects.**	**31**
Exp6.15	**Create two interfaces Printer and Scanner. Design class AllInOnePrinter implementing both interfaces. Demonstrate multiple inheritance using interfaces.**	**33**
Exp6.16	**Design an abstract class Appliance with method powerConsumption(). Implement subclasses Fan, AC, and Refrigerator. Compute total power consumption using dynamic binding.**	**35**
Exp6.17	**Design a Bank Transaction System:** **Abstract class → Account** **Interfaces → Loan, Insurance** **Classes → SavingsAccount, CurrentAccount** **The program should demonstrate:** **Multiple inheritance using interfaces** **Runtime polymorphism** **Array of objects** **Menu-driven execution**	**37**

Th1)

Q. Why is Runtime Polymorphism also called "Late Binding"?

A. Runtime Polymorphism can also be called 'Late Binding'. This is because, in Runtime Polymorphism, the method call is not bound to the method definition at compile time, but at runtime only. When a child object is assigned to a parent reference, if an overridden method is called, the compiler does not know which version to execute because all it can see is the reference type. It is only at runtime, when the JVM knows what the object type is, that the method call is bound to the appropriate method, hence the term 'Late Binding'.

Th2)

Q. Can you override a static method in Java? Why or why not?

A. Static methods in Java cannot be overridden because they are not associated with any object but with the class. Therefore, static methods in Java are executed at compile time using the reference type, which is known as static binding or early binding. If we define any method with the same signature in any of its subclasses, then it is not overriding but method hiding. The difference is that in overriding, we resolve the method call at runtime using the actual object, but in static methods, even if we have an object of the child class in a reference of the parent class, it will always call the parent class method because it is executed at compile time. Therefore, since overriding is associated with dynamic dispatch, which is executed at runtime, static methods cannot be overridden in Java.

Th3)

Q . What is the difference between this() and super() calls?

A. this() is used to call other constructors of the same class, which is helpful when different constructors have similar code and can be reused. super() is used to call the parent class constructors from the child class, similar to when SBI and HDFC used super(p, si, 3.2) to set the values of the Bank class fields. These

keywords should be the first line of code in the constructors and cannot be used together in the same constructor. If super() is not used in the constructors, Java will automatically insert a default super() call, but only if the parent class has a default constructor; otherwise, it should be done manually with the appropriate parameters.

Th4)

Q. What happens if you try to override a final method?

A. If you try to override a final method, you get a compile-time error, meaning Java itself won't allow you to run your program. By making a method final, you're informing the compiler that this is the final version of this method, which can't be overridden in any of your subclasses. If you define a method in your subclass with the same signature as an existing method in your superclass, you'll get an immediate "cannot override final method" error from the compiler. Similarly, you can make a class final, such as Java's String, meaning you can't subclass it at all. So, final on a method means it is closed for overriding.

Th5)

Q. Explain Covariant Return Types in the context of method overriding.

A. Normally, in order for an overriding method to be allowed, it has to have the exact same return type as the parent's method. However, there is an exception to this rule, and that is covariant return type. This allows the child's method to return a subtype of the parent's method's return type. So if the parent method returns a Bank, then the child's method is allowed to override it and return an SBI, as SBI is also a Bank, it is just more specific. This feature was introduced in Java 5. The logic is clean. The child is still within its rights if it says, "I'll return an SBI," as it is saying, "I'll return a Bank," and that is perfectly valid. There is also no need for casting. One thing that is important to note is that it is only allowed for it to be narrower (i.e., more specific), not broader. So if the parent's method returns an SBI, then it is not allowed for the child's method to override it and return a Bank, as that is not allowed in Java.

Th6)

Q. Is it possible to perform multiple inheritance using classes in Java?

A. Multiple inheritance using classes is not allowed in Java, meaning a class cannot inherit from two or more classes at the same time. This is because of the diamond problem. For instance, if we have two classes, A and B, both of which are inherited by a class C, and both classes contain a method with the same

name, Java gets confused about which method to use in class C. To eliminate this ambiguity, Java does not allow a class to inherit from two classes. Multiple inheritance is, however, possible using interfaces in Java, meaning a class can implement any number of interfaces. This was also creating the diamond problem in Java, but with Java 8, the default methods in interfaces have made it possible again, though Java forces the programmer to override the method in the class.

Th7)

Q. Can abstract classes have constructors?

A. Yes, abstract classes can have constructors, although you cannot directly use them to create an abstract class object. The reason for this is that an abstract class constructor is called by the constructor of the child class using super(), either explicitly or implicitly. The purpose of an abstract class constructor is not to create an abstract class object, but to initialize the abstract class's fields as part of the child class object. For instance, in our Bank program, we needed to initialize the fields p, si, and r, which was done by the abstract class constructor through super() itself, as in the case of creating an SBI or HDFC object. It is a compile-time error to directly use new AbstractClass(), but indirectly, an abstract class constructor is called every time a child class object is created.

Th8)

Q. What are Default methods?

A. Default methods in Java are interface methods that have an actual body. They were introduced in Java 8. Prior to Java 8, all interface methods were abstract. Adding any new method in an interface would have broken all classes that implemented that interface. This is where default methods shine. A default method is defined in an interface with the default keyword. It is implemented in full. All classes that implement that interface inherit that method. However, there is no need for them to implement it. They can if they wish, but it is not mandatory. The problem with the diamond method is that if two interfaces have default methods with the same name, then if a class implements both of them, Java will still force it to implement that method. If not, then it is a compile-time error. Therefore, default methods in Java are simply a feature of backward compatibility.

Th9)

Q. Is 'final' allowed for abstract classes?

A. final and abstract are polar opposites of each other and can't be combined. The purpose of an abstract class is that it needs to be extended. It's not meant to be used on its own and requires the derived classes to implement the abstract methods. The purpose of a final class, on the other hand, is that it's not meant to be extended. So, if you declare a class as final abstract class Something, you're saying, "This class needs to be extended, but this class can't be extended." That's a logical contradiction, and Java will throw a compilation error right away. The same logic applies to methods as well. The purpose of declaring a method as abstract is that it needs to be overridden in the derived classes, but the purpose of declaring a method as final is that it can't be overridden. So, abstract final on a method declaration is not allowed. The rule of thumb is that final and abstract can't go hand in hand, not on a class or a method.

Th10)

Q. Difference between extends & implements.

A. In Java, both extends and implements are used for inheritance, but they are applied in different situations. The extends keyword is used to connect one class to another class or to link one interface to another interface. With this, you inherit both the method declarations and their complete implementations from the parent.

On the other hand, implements is used when a class adopts an interface. A class can implement multiple interfaces, but it only gets the method declarations, meaning you have to provide the implementation yourself unless the interface includes default methods.

So, extends gives you ready-made functionality from a single source, while implements allows you to adopt multiple interfaces where you define the actual behavior.

Th11)

Q. What are Marker Interfaces?

A. "Marker interface in Java is basically an interface with no methods and no fields, and its only purpose is to be used as a flag to indicate to the JVM or

compiler that the class is to be treated in some particular way. For example, implementing the Serializable interface is essentially informing the JVM that it is safe to convert the class into a byte stream, and implementing Cloneable is informing the JVM that the class is safe to be cloned. If you do not do this, the JVM will not allow serialization or cloning of the class and will throw an exception when you try to do so."

Although annotations have now been added in Java 5, which is arguably more powerful and flexible in modern times, marker interfaces are still an integral part of Java and should be grasped as an essential concept.

Th12)

Q. Can an interface extend another interface?

A. Yes, a Java interface can extend one or more interfaces. The way to do this is by utilizing the "extends" keyword. When a child interface extends a parent interface, all the method signatures of the parent are automatically implemented in the child, and any class that may implement the child interface in the future will have to implement all of them. As a result of this, since conventional interfaces are simply passing on the method name and not the actual implementation, Java allows for a single interface to extend a number of other interfaces without any "diamond problem" that may result from a class. However, if there are conflicting default methods in a number of the parent interfaces, since default methods contain code, the final class that implements them will be forced to implement that particular default method.

Th13)

Q. Can we define static methods in interfaces?

A. Yes, Java has supported static methods in interfaces since Java 8. Although they are full-written code, just like default methods, they are part of the interface itself and are never inherited by the classes that implement the interface or the sub-interfaces. As such, they must be called directly using the name of the interface (e.g., InterfaceName.methodName()). They are essentially just a way to store utility or "helper" functions, such as a general calculator in a Bank interface, keeping all the tools related to the interface organized without cluttering the classes that implement the interface.

Th14)

Q. Define 'Loose Coupling'.

A. Loose Coupling is all about making your classes independent from each other by making them dependent only on interfaces (like Bank), not their implementations (like SBI). This ensures that you are not hard-coding them together in any way. If you ever want to switch from SBI to HDFC, you can do so without affecting the rest of your codebase. Programming against these abstractions makes your code super easy to maintain, test, and scale without breaking it.

Exp 6.1

```
//Write a Java program to demonstrate single inheritance using class Person
and subclass Student Override a method display() in the subclass
class Person
{
    void display()
    {
        System.out.println("This is a person.");
    }
}
class Student extends Person
{
    void display()
    {
        System.out.println("This is a student.");
    }
}
public class exe6_1
{
    public static void main(String[] args)
    {
        Person p=new Person();
        p.display();
        Student s=new Student();
        s.display();
    }
}
```

```
OUTPUT 45    TERMINAL    PORTS

PS C:\Users\<Anmol> > cd oOPS/exe6
PS C:\Users\<Anmol> > javac exe6_1.java
PS C:\Users\<Anmol> > java exe6_1
This is a person.
This is a student.
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have a superclass Person with a method display() that prints "This is a person." The subclass Student extends Person and overrides the display() method to print "This is a student." In the main method, we create an instance of Person and call the display() method, which outputs "This is a person." Then we create an instance of Student and call the display() method, which outputs "This is a student." This demonstrates single inheritance and method overriding in Java.

Exp 6.2

```
//Develop a Java program to implement runtime polymorphism using class Shape
and subclass Circle.
 Override the method area().
class Shape
{
    void area()
    {
        System.out.println("Area of shape");
    }
}
class Circle extends Shape
{
    void area()
    {
        System.out.println("Area of circle");
    }
}
public class exe6_2
{
    public static void main(String[] args)
    {
        Shape s=new Shape();
```

```
        s.area();
        Circle c=new Circle();
        c.area();
    }
}
```

```
PS C:\Users\<Anmol> > javac exe6_2.java
PS C:\Users\<Anmol> > java exe6_2
Area of shape
Area of circle
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have a superclass Shape with a method area() that prints "Area of shape." The subclass Circle extends Shape and overrides the area() method to print "Area of circle." In the main method, we create an instance of Shape and call the area() method, which outputs "Area of shape." Then we create an instance of Circle and call the area() method, which outputs "Area of circle." This demonstrates runtime polymorphism and method overriding in Java as the method that gets called is determined at runtime based on the object type.

Exp 6.3

```
//Write a Java program to demonstrate hierarchical inheritance using class
Vehicle and subclasses Car and Bike.
class Vehicle
{
    void display()
    {
        System.out.println("This is a vehicle.");
    }
}
class Car extends Vehicle
{
    void display()
    {
        System.out.println("This is a car.");
    }
}
class Bike extends Vehicle
{
    void display()
```

```
    {
        System.out.println("This is a bike.");
    }
}
public class exe6_3
{
    public static void main(String[] args)
    {
        Vehicle v=new Vehicle();
        v.display();
        Car c=new Car();
        c.display();
        Bike b=new Bike();
        b.display();
    }
}
```

```
PS C:\Users\<Anmol> > javac exe6_3.jav
PS C:\Users\<Anmol> > java exe6_3
This is a vehicle.
This is a car.
This is a bike.
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have a superclass Vehicle with a method display() that prints "This is a vehicle." The subclasses Car and Bike extend Vehicle and override the display() method to print "This is a car." and "This is a bike." respectively. In the main method, we create an instance of Vehicle and call the display() method, which outputs "This is a vehicle." Then we create an instance of Car and call the display() method, which outputs "This is a car." Finally, we create an instance of Bike and call the display() method, which outputs "This is a bike." This demonstrates hierarchical inheritance in Java as both Car and Bike inherit from the same superclass Vehicle.

Exp 6.4

```
// Implement a Java program to demonstrate method overriding using class Bank
and subclasses SBI and HDFC to compute interest rates
import java.util.Scanner;
class Bank
{
    double p,si,r,t;
```

```
    Bank(double p,double si,double t)
    {
        this.p=p;
        this.si=si;
        this.t=t;
        r=(si*100)/(p*t);
    }
    void interest()
    {
        System.out.println("Interest rate of bank is:"+r+"% for time: "+t+"
years");
    }
}
class SBI extends Bank
{
    SBI(double p,double si)
    {
        super(p,si,3.2);
    }
    void interest()
    {
        System.out.println("Interest rate of SBI is: "+r+"% for time: "+t+"
years");
    }
}
class HDFC extends Bank
{
    HDFC(double p,double si)
    {
        super(p,si,5.6);
    }
    void interest()
    {
        System.out.println("Interest rate of HDFC is: "+r+"% for time: "+t+"
years");
    }
}
public class exe6_4
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter principal amount: ");
        double p=sc.nextDouble();
        System.out.print("Enter simple interest: ");
        double si=sc.nextDouble();
        Bank b=new Bank(p,si,2);
        b.interest();
```

```
        SBI s=new SBI(p,si);
        s.interest();
        HDFC h=new HDFC(p,si);
        h.interest();
    }
}
```

```
PS C:\Users\<Anmol> > javac exe6_4.java
PS C:\Users\<Anmol> > java exe6_4
Enter principal amount: 1547.56
Enter simple interest: 9856.32
Interest rate of bank is:318.4471038279614% for time: 2.0 years
Interest rate of SBI is: 199.0294398924759% for time: 3.2 years
Interest rate of HDFC is: 113.73110850998624% for time: 5.6 years
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have a superclass Bank with a constructor that calculates the interest rate based on the principal amount (p) and simple interest (si) taken as user input. The subclasses SBI and HDFC extend Bank and override the interest() method to display the interest rates specific to each bank using their fixed time values (3.2 and 5.6 respectively). In the main method, we take p and si from the user via Scanner, create instances of Bank, SBI, and HDFC with those values, and call their interest() methods. This demonstrates method overriding in Java with dynamic user input.

Exp 6.5

```
//Write a Java program to demonstrate multilevel inheritance using classes
Employee → Manager → Director.
class Employee
{
    void display()
    {
        System.out.println("This is an employee.");
    }
}
class Manager extends Employee
{
    void display()
```

```java
    {
        System.out.println("This is a manager.");
    }
}
class Director extends Manager
{
    void display()
    {
        System.out.println("This is a director.");
    }
}
public class exe6_5
{
    public static void main(String[] args)
    {
        Employee e=new Employee();
        e.display();
        Manager m=new Manager();
        m.display();
        Director d=new Director();
        d.display();
    }
}
```

```
OUTPUT 49    TERMINAL    PORTS

PS C:\Users\<Anmol> > javac exe6_5.java
PS C:\Users\<Anmol> > java exe6_5
This is an employee.
This is a manager.
This is a director.
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have a superclass Employee with a method display() that prints "This is an employee." The subclass Manager extends Employee and overrides the display() method to print "This is a manager." The subclass Director extends Manager and overrides the display() method to print "This is a director." In the main method, we create an instance of Employee and call the display() method, which outputs "This is an employee." Then we create an instance of Manager and call the display() method, which outputs "This is a manager." Finally, we create an instance of Director and call the display() method, which outputs "This is a director." This demonstrates multilevel inheritance and method overriding in Java.

Exp 6.6

```
//Develop a Java program to demonstrate runtime polymorphism using an array of
objects. Use class Animal and subclasses Dog and Cat.
class Animal
{
    void sound()
    {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal
{
    void sound()
    {
        System.out.println("Dog barks");
    }
}
class Cat extends Animal
{
    void sound()
    {
        System.out.println("Cat meows");
    }
}
public class exe6_6
{
    public static void main(String args[])
    {
        Animal[] animals=new Animal[3];
        animals[0]=new Animal();
        animals[1]=new Dog();
        animals[2]=new Cat();
        for (Animal animal:animals)
        {
            animal.sound();
        }
    }
}
```

```
PS C:\Users\<Anmol> > javac exe6_6.java
PS C:\Users\<Anmol> > java exe6_6
Animal makes a sound
Dog barks
Cat meows
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have a superclass Animal with a method sound() that prints "Animal makes a sound." The subclasses Dog and Cat extend Animal and override the sound() method to print "Dog barks" and "Cat meows" respectively. In the main method, we create an array of Animal objects and assign instances of Animal, Dog, and Cat to it. We then use a for-each loop to iterate through the array and call the sound() method on each object. This demonstrates runtime polymorphism in Java, as the method that gets called is determined at runtime based on the actual object type (Animal, Dog, or Cat) rather than the reference type (Animal).

Exp 6.7

```
//Write a Java program to demonstrate dynamic method dispatch using user
input. Create base class Shape and subclasses Circle and Square.
import java.util.Scanner;
class Shape
{    void area()
    {
        System.out.println("Area of shape");
    }
}
class Circle extends Shape
{    void area()
    {
        System.out.println("Area of circle");
    }
}
class Square extends Shape
{    void area()
    {
        System.out.println("Area of square");
    }
}
public class exe6_7
{
```

```java
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter shape (circle/square): ");
        String shape=sc.nextLine();
        Shape s;
        if(shape.equalsIgnoreCase("circle"))
        {
            s=new Circle();
            s.area();
        }
        else if(shape.equalsIgnoreCase("square"))
        {
            s=new Square();
            s.area();
        }
        else
        {
            System.out.println("Invalid shape");
        }
    }
}
```

```
OUTPUT 50    TERMINAL    PORTS

PS C:\Users\<Anmol> > javac exe6_7.java
PS C:\Users\<Anmol> > java exe6_7
Enter shape (circle/square): circle
Area of circle
PS C:\Users\<Anmol> > java exe6_7
Enter shape (circle/square): kjh
Invalid shape
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have a superclass Shape with a method area() that prints "Area of shape." The subclasses Circle and Square extend Shape and override the area() method to print "Area of circle" and "Area of square" respectively. In the main method, we take user input to determine which shape's area method to call. Based on the user's input, we create an instance of either Circle or Square and call the area() method, demonstrating dynamic method dispatch in Java. If the user enters "circle", it will output "Area of circle". If the user enters "square", it will output "Area of square". If the user enters an invalid shape, it will output "Invalid shape". This shows how the

method that gets called is determined at runtime based on the object type created from user input.

Exp 6.8

```
//Design a Java program to demonstrate method overriding using the super
keyword.  Create base class Vehicle and subclass Car where overridden method
calls parent method.
class Vehicle
{
    void display()
    {
        System.out.println("This is a vehicle.");
    }
}
class Car extends Vehicle
{
    void display()
    {
        super.display();
        System.out.println("This is a car.");
    }
}
public class exe6_8
{
    public static void main(String[] args)
    {
        Car c=new Car();
        c.display();
    }
}
```

```
PS C:\Users\<Anmol> > javac exe6_8.java
PS C:\Users\<Anmol> > java exe6_8
This is a vehicle.
This is a car.
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have a superclass Vehicle with a method display() that prints "This is a vehicle." The subclass Car extends Vehicle and overrides the display() method to first call the parent method using super.display(), which outputs "This is a vehicle." Then it prints "This is a car."

In the main method, we create an instance of Car and call the display() method, which demonstrates method overriding and the use of the super keyword to call the parent class method in Java.

Exp 6.9

```
// Create an abstract class Employee with abstract method calculateSalary().
Implement subclasses Manager and Developer and override the method.
import java.util.Scanner;
abstract class Employee
{
    abstract double calculateSalary();
}
class Manager extends Employee
{
    double baseSalary;
    Manager(double baseSalary)
    {
        this.baseSalary=baseSalary;
    }
    double calculateSalary()
    {
        return baseSalary+5000;
    }
}
class Developer extends Employee
{
    double baseSalary;
    Developer(double baseSalary)
    {
        this.baseSalary=baseSalary;
    }
    double calculateSalary()
    {
        return baseSalary+3000;
    }
}
public class exe6_9
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter manager base salary: ");
        double ms=sc.nextDouble();
        System.out.print("Enter developer base salary: ");
        double ds=sc.nextDouble();
```

```
        Employee manager=new Manager(ms);
        Employee developer=new Developer(ds);
        System.out.println("Manager Salary: "+manager.calculateSalary());
        System.out.println("Developer Salary: "+developer.calculateSalary());
    }
}
```

```
OUTPUT 52    TERMINAL    PORTS

PS C:\Users\<Anmol> > javac exe6_9.java
PS C:\Users\<Anmol> > java exe6_9
Enter manager base salary: 56002
Enter developer base salary: 7895.65
Manager Salary: 61002.0
Developer Salary: 10895.65
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have an abstract class Employee with an abstract method calculateSalary(). The subclasses Manager and Developer extend Employee and override calculateSalary() to return the base salary plus their respective bonuses of 5000 and 3000. In the main method, base salaries are taken as user input via Scanner, and Employee references are used to hold Manager and Developer objects, demonstrating runtime polymorphism along with abstract classes.

Exp 6.10

```
// Design an interface Shape with method area(). Implement classes Circle,
Rectangle, and Triangle.
import java.util.Scanner;
interface Shape
{
    double area();
}
class Circle implements Shape
{
    double radius;
    Circle(double radius)
    {
        this.radius=radius;
```

```
    }
    public double area()
    {
        return Math.PI*radius*radius;
    }
}
class Rectangle implements Shape
{
    double length,width;
    Rectangle(double length,double width)
    {
        this.length=length;
        this.width=width;
    }
    public double area()
    {
        return length*width;
    }
}
class Triangle implements Shape
{
    double base,height;
    Triangle(double base,double height)
    {
        this.base=base;
        this.height=height;
    }
    public double area()
    {
        return 0.5*base*height;
    }
}
public class exe6_10
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter radius of circle: ");
        double r=sc.nextDouble();
        System.out.print("Enter length and width of rectangle: ");
        double l=sc.nextDouble();
        double w=sc.nextDouble();
        System.out.print("Enter base and height of triangle: ");
        double b=sc.nextDouble();
        double h=sc.nextDouble();
        Shape circle=new Circle(r);
        Shape rectangle=new Rectangle(l,w);
        Shape triangle=new Triangle(b,h);
```

```
        System.out.println("Area of Circle: "+circle.area());
        System.out.println("Area of Rectangle: "+rectangle.area());
        System.out.println("Area of Triangle: "+triangle.area());
    }
}
```

```
PS C:\Users\<Anmol> > javac exe6_10.java
PS C:\Users\<Anmol> > java exe6_10
Enter radius of circle: 5.9
Enter length and width of rectangle: 8.3
4.6
Enter base and height of triangle: 8.9
7.1
Area of Circle: 109.35884027146072
Area of Rectangle: 38.18
Area of Triangle: 31.595
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have an interface Shape with a method area(). The classes Circle, Rectangle, and Triangle implement the Shape interface and provide their own implementation of the area() method. In the main method, we take user input for the dimensions of each shape and create instances of Circle, Rectangle, and Triangle using Shape references. We then call the area() method on each shape, demonstrating polymorphism through interfaces. Each shape calculates its area based on its specific formula, and the results are printed to the console.

Exp 6.11

```
//Create an abstract class Vehicle with abstract method start() and Implement
subclasses Car and Bike demonstrating runtime polymorphism
import java.util.Scanner;
abstract class Vehicle
{
    abstract void start();
}
class Car extends Vehicle
{    void start()
    {
```

```
        System.out.println("Car is starting with a key.");
    }
}
class Bike extends Vehicle
{
    void start()
    {
        System.out.println("Bike is starting with a button.");
    }
}
public class exe6_11
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter vehicle (car/bike): ");
        String vehicle=sc.nextLine();
        Vehicle v;
        if(vehicle.equalsIgnoreCase("car"))
        {
            v=new Car();
            v.start();
        }
        else if(vehicle.equalsIgnoreCase("bike"))
        {
            v=new Bike();
            v.start();
        }
        else
        {
            System.out.println("Invalid vehicle");
        }
    }
}
```

```
OUTPUT 51    TERMINAL    PORTS

PS C:\Users\<Anmol> > javac exe6_11.java
PS C:\Users\<Anmol> > java exe6_11
Enter vehicle (car/bike): car
Car is starting with a key.
PS C:\Users\<Anmol> > java exe6_11
Enter vehicle (car/bike): hjbhj
Invalid vehicle
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have an abstract class Vehicle with an abstract method start(). The subclasses Car and Bike extend Vehicle and override the start() method to print a specific message for each vehicle. In the main method, we take user input to determine which vehicle's start method to call. Based on the user's input, we create an instance of either Car or Bike and call the start() method, demonstrating runtime polymorphism in Java with user input

Exp 6.12

```
//Write a Java program to demonstrate interface-based runtime polymorphism
using Payment interface and classes CreditCard and UPI
import java.util.Scanner;
interface Payment
{
    void pay(double amount);
}
class CreditCard implements Payment
{
    public void pay(double amount)
    {
        System.out.println("Paid "+amount+" using Credit Card.");
    }
}
class UPI implements Payment
{
    public void pay(double amount)
    {
        System.out.println("Paid "+amount+" using UPI.");
    }
}
```

```java
public class exe6_12
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter payment method (credit/upi): ");
        String method=sc.nextLine();
        System.out.print("Enter amount to pay: ");
        double amount=sc.nextDouble();
        Payment payment;
        if(method.equalsIgnoreCase("credit"))
        {
            payment=new CreditCard();
            payment.pay(amount);
        }
        else if(method.equalsIgnoreCase("upi"))
        {
            payment=new UPI();
            payment.pay(amount);
        }
        else
        {
            System.out.println("Invalid payment method");
        }
    }
}
```

```
OUTPUT 53    TERMINAL    PORTS

PS C:\Users\<Anmol> > javac exe6_12.java
PS C:\Users\<Anmol> > java exe6_12
Enter payment method (credit/upi): upi
Enter amount to pay: 5946
Paid 5946.0 using UPI.
PS C:\Users\<Anmol> > java exe6_12
Enter payment method (credit/upi): 65456
Enter amount to pay: 465465
Invalid payment method
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have an interface Payment with a method pay(). The classes CreditCard and UPI implement the Payment interface and provide their own implementation of the pay() method. In the main method, we take user input to determine which payment method to use and the amount to pay. Based on the user's input, we create an instance of either CreditCard or UPI

using Payment references and call the pay() method, demonstrating interface-based runtime polymorphism in Java with user input.

Exp 6.13

```
//Create an interface Drawable and demonstrate hierarchical implementation
 using classes Circle and Square.
import java.util.Scanner;
interface Drawable
{
    void draw();
}
class Circle implements Drawable
{
    public void draw()
    {
        System.out.println("Drawing a Circle.");
    }
}
class Square implements Drawable
{
    public void draw()
    {
        System.out.println("Drawing a Square.");
    }
}
public class exe6_13
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter shape to draw (circle/square): ");
        String shape=sc.nextLine();
        Drawable d;
        if(shape.equalsIgnoreCase("circle"))
        {
            d=new Circle();
            d.draw();
        }
        else if(shape.equalsIgnoreCase("square"))        {
            d=new Square();
            d.draw();
        }
        else
        {
            System.out.println("Invalid shape");
```

```
        }
    }
}
```

```
PS C:\Users\<Anmol> > javac exe6_13.java
PS C:\Users\<Anmol> > java exe6_13
Enter shape to draw (circle/square): square
Drawing a Square.
PS C:\Users\<Anmol> > java exe6_13
Enter shape to draw (circle/square): shyt
Invalid shape
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have an interface Drawable with a method draw(). The classes Circle and Square implement the Drawable interface and provide their own implementation of the draw() method. In the main method, we take user input to determine which shape to draw. Based on the user's input, we create an instance of either Circle or Square using Drawable references and call the draw() method, demonstrating hierarchical implementation of interfaces in Java with user input. Each shape's draw() method prints a specific message to the console, indicating which shape is being drawn.

Exp 6.14

```
// Design an abstract class Bank with method getInterestRate(). Implement
subclasses SBI, HDFC, and ICICI. Demonstrate runtime polymorphism using array
of objects.
import java.util.Scanner;
abstract class Bank
{
    double p,si,r,t;
    Bank(double p,double si,double t)
    {
        this.p=p;
        this.si=si;
        this.t=t;
        r=(si*100)/(p*t);
    }
    abstract void getInterestRate();
```

```
}
class SBI extends Bank
{
    SBI(double p,double si)
    {
        super(p,si,3.2);
    }
    public void getInterestRate()
    {
        System.out.println("Interest rate of SBI is: "+r+"% for time: "+t+"
years");
    }
}
class HDFC extends Bank
{
    HDFC(double p,double si)
    {
        super(p,si,5.6);
    }
    public void getInterestRate()
    {
        System.out.println("Interest rate of HDFC is: "+r+"% for time: "+t+"
years");
    }
}
class ICICI extends Bank
{
    ICICI(double p,double si)
    {
        super(p,si,4.1);
    }
    public void getInterestRate()
    {
        System.out.println("Interest rate of ICICI is: "+r+"% for time: "+t+"
years");
    }
}
public class exe6_14
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        System.out.print("Enter principal amount: ");
        double p=sc.nextDouble();
        System.out.print("Enter simple interest: ");
        double si=sc.nextDouble();
        Bank b[]=new Bank[3];
        b[0]=new SBI(p,si);
```

```
        b[1]=new HDFC(p,si);
        b[2]=new ICICI(p,si);
        for(int i=0;i<b.length;i++)
        {
            b[i].getInterestRate();
        }
    }
}
```

```
PS C:\Users\<Anmol> > javac exe6_14.java
PS C:\Users\<Anmol> > java exe6_14
Enter principal amount: 895476
Enter simple interest: 1450264
Interest rate of SBI is: 50.6107924723834% for time: 3.2 years
Interest rate of HDFC is: 28.920452841361946% for time: 5.6 years
Interest rate of ICICI is: 39.501106319909% for time: 4.1 years
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have an abstract class Bank with a constructor that calculates the interest rate based on user input principal amount (p) and simple interest (si). The subclasses SBI, HDFC, and ICICI extend Bank and override getInterestRate() to display their respective interest rates using fixed time values of 3.2, 5.6, and 4.1 years. In the main method, we take p and si from the user via Scanner, create an array of Bank references holding SBI, HDFC, and ICICI objects, and iterate over it calling getInterestRate() on each element. This demonstrates runtime polymorphism as the correct overridden method is called based on the actual object type at runtime.

Exp 6.15

```
// Create two interfaces Printer and Scanner. Design class AllInOnePrinter
implementing both interfaces. Demonstrate multiple inheritance using
interfaces.
interface Printer
{
    void print();
}
interface Scanner
{
    void scan();
```

```java
}
class AllInOnePrinter implements Printer,Scanner
{
    public void print()
    {
        System.out.println("Printing document...");
    }
    public void scan()
    {
        System.out.println("Scanning document...");
    }
}
public class exe6_15
{
    public static void main(String[] args)
    {
        AllInOnePrinter aip=new AllInOnePrinter();
        aip.print();
        aip.scan();
    }
}
```

```
PS C:\Users\<Anmol> > javac exe6_15.java
PS C:\Users\<Anmol> > java exe6_15
Printing document...
Scanning document...
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have two interfaces Printer and Scanner with methods print() and scan() respectively. The class AllInOnePrinter implements both interfaces and provides implementations for both methods. In the main method, we create an instance of AllInOnePrinter and call both print() and scan() methods, demonstrating multiple inheritance using interfaces in Java. The output will show messages indicating that the document is being printed and scanned, showcasing the functionality of both interfaces in a single class.

Exp 6.16

```
//Design an abstract class Appliance with method powerConsumption()Implement
subclasses Fan, AC, and RefrigeratorCompute total power consumption using
dynamic binding.
import java.util.Scanner;
abstract class Appliance
{
    abstract double powerConsumption();
}
class Fan extends Appliance
{    double power;
    Fan(double power)
    {
        this.power=power;
    }
    double powerConsumption()
    {
        return power;
    }
}
class AC extends Appliance
{    double power;
    AC(double power)
    {
        this.power=power;
    }
    double powerConsumption()
    {
        return power;
    }
}
class Refrigerator extends Appliance
{    double power;
    Refrigerator(double power)
    {
        this.power=power;
    }
    double powerConsumption()
    {
        return power;
    }
}
public class exe6_16
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
```

```java
        System.out.print("Enter power consumption of fan: ");
        double fanPower=sc.nextDouble();
        System.out.print("Enter power consumption of AC: ");
        double acPower=sc.nextDouble();
        System.out.print("Enter power consumption of refrigerator: ");
        double fridgePower=sc.nextDouble();
        Appliance appliances[]=new Appliance[3];
        appliances[0]=new Fan(fanPower);
        appliances[1]=new AC(acPower);
        appliances[2]=new Refrigerator(fridgePower);
        double totalPower=0;
        for(Appliance a: appliances)
        {
            totalPower+=a.powerConsumption();
        }
        System.out.println("Total power consumption: "+totalPower+" watts");
    }
}
```

```
OUTPUT 53    TERMINAL    PORTS

PS C:\Users\<Anmol> > javac exe6_16.java
PS C:\Users\<Anmol> > java exe6_16
Enter power consumption of fan: 8965
Enter power consumption of AC: 425
Enter power consumption of refrigerator: 789
Total power consumption: 10179.0 watts
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have an abstract class Appliance with an abstract method powerConsumption(). The subclasses Fan, AC, and Refrigerator extend Appliance and override powerConsumption() to return their respective power consumption values. In the main method, we take user input for the power consumption of each appliance, create an array of Appliance references holding Fan, AC, and Refrigerator objects, and iterate over it to calculate the total power consumption. This demonstrates runtime polymorphism as the correct overridden method is called based on the actual object type at runtime, allowing us to compute the total power consumption dynamically.

Exp 6.17

```
//Design a Bank Transaction System:
// Abstract class → Account
// Interfaces → Loan, Insurance
// Classes → SavingsAccount, CurrentAccount
// The program should demonstrate:
// Multiple inheritance using interfaces
// Runtime polymorphism
// Array of objects
// Menu-driven execution
import java.util.Scanner;
abstract class Account
{
    double bal;
    Account(double b)
    {
        bal=b;
    }
    abstract void dep(double a);
    abstract void wit(double a);
}
interface Loan
{
    void loan(double a);
}
interface Insurance
{
    void ins(double a);
}
class SavingsAccount extends Account implements Loan
{
    SavingsAccount(double b)
    {
        super(b);
    }
    void dep(double a)
    {
        bal+=a;
    }
    void wit(double a)
    {
        if(a<=bal) bal-=a;
        else System.out.println("Insufficient");
    }
    public void loan(double a)
    {
        System.out.println("Loan "+a+" applied");
```

```
    }
}
class CurrentAccount extends Account implements Insurance
{
    CurrentAccount(double b)
    {
        super(b);
    }
    void dep(double a)
    {
        bal+=a;
    }
    void wit(double a)
    {
        if(a<=bal) bal-=a;
        else System.out.println("Insufficient");
    }
    public void ins(double a)
    {
        System.out.println("Insurance "+a+" bought");
    }
}
public class exe6_17
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);
        Account acc[]=new Account[2];
        acc[0]=new SavingsAccount(0);
        acc[1]=new CurrentAccount(0);
        int ch,i;
        double a;
        while(true)
        {
            System.out.println("\n1.Deposit 2.Withdraw 3.Loan 4.Insurance
5.Show 6.Exit");
            System.out.print("Enter choice: ");
            ch=sc.nextInt();
            if(ch==6) break;
            System.out.print("Account (1/2): ");
            i=sc.nextInt()-1;
            if(i<0||i>1)
            {
                System.out.println("Invalid account");
                continue;
            }
            switch(ch)
            {
```

```
                case 1:
                    System.out.print("Amount: ");
                    a=sc.nextDouble();
                    acc[i].dep(a);
                    break;
                case 2:
                    System.out.print("Amount: ");
                    a=sc.nextDouble();
                    acc[i].wit(a);
                    break;
                case 3:
                    if(acc[i] instanceof SavingsAccount)
                    {
                        System.out.print("Loan amt: ");
                        a=sc.nextDouble();
                        ((SavingsAccount)acc[i]).loan(a);
                    }
                    else System.out.println("Not allowed");
                    break;
                case 4:
                    if(acc[i] instanceof CurrentAccount)
                    {
                        System.out.print("Insurance amt: ");
                        a=sc.nextDouble();
                        ((CurrentAccount)acc[i]).ins(a);
                    }
                    else System.out.println("Not allowed");
                    break;
                case 5:
                    System.out.println("Balance: "+acc[i].bal);
                    break;
                default:
                    System.out.println("Invalid");
            }
        }
    }
}
```

```
OUTPUT 55    TERMINAL    PORTS

PS C:\Users\<Anmol> > javac exe6_17.java
PS C:\Users\<Anmol> > java exe6_17

1.Deposit 2.Withdraw 3.Loan 4.Insurance 5.Show 6.Exit
Enter choice: 1
Account (1/2): 1
Amount: 1000

1.Deposit 2.Withdraw 3.Loan 4.Insurance 5.Show 6.Exit
Enter choice: 5
Account (1/2): 1
Balance: 2000.0

1.Deposit 2.Withdraw 3.Loan 4.Insurance 5.Show 6.Exit
Enter choice: 2
Account (1/2): 1
Amount: 1000

1.Deposit 2.Withdraw 3.Loan 4.Insurance 5.Show 6.Exit
Enter choice: 5
Account (1/2): 1
Balance: 1000.0

1.Deposit 2.Withdraw 3.Loan 4.Insurance 5.Show 6.Exit
Enter choice: 6
PS C:\Users\<Anmol> >
```

Observation: In the above code, we have an abstract class Account with a constructor that initializes the balance (bal) and abstract methods dep() and wit() for deposit and withdrawal. We have two interfaces, Loan and Insurance, with methods loan() and ins() respectively. The SavingsAccount class extends Account and implements Loan, while the CurrentAccount class extends Account and implements Insurance. In the main method, we create an array of Account objects to hold instances of SavingsAccount and CurrentAccount. We then use a menu-driven approach to perform various operations like deposit, withdraw, apply for a loan, buy insurance, and show balance. This demonstrates multiple inheritance using interfaces, runtime polymorphism through method overriding, and the use of an array of objects to manage different account types in a bank transaction system in Java.